

Descripción de la Persistencia de Datos

Resumen

La solución implementa persistencia mediante **JPA (Java Persistence API) con Hibernate** como ORM. Cada microservicio tiene su propia base de datos independiente (patrón Database Per Service), asegurando escalabilidad horizontal y desacoplamiento.

Componentes de persistencia

estudiante-service (Puerto: 8081)

- **Tecnología:** JPA / Hibernate
- **Base de datos:** MySQL (configurable vía ENV vars)
- **Tabla principal:** estudiantes

asistencia-service (Puerto: 8082)

- **Tecnología:** JPA / Hibernate
- **Base de datos:** MySQL (configurable vía ENV vars) o H2
- **Tabla principal:** asistencias

evaluacion-service (Puerto: 8083)

- **Tecnología:** JPA / Hibernate
- **Base de datos:** MySQL (configurable vía ENV vars)
- **Tabla principal:** evaluaciones

bff-service (Puerto: 8084)

- **Sin persistencia directa:** Solo composición de servicios (patrón BFF)
- **Cache:** Puede almacenar localmente si es requerido (no implementado)

Esquema de base de datos por microservicio

estudiante-service - Tabla estudiantes

```
CREATE TABLE estudiantes (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  run VARCHAR(12) NOT NULL UNIQUE,  
  nombre VARCHAR(255) NOT NULL,  
  curso VARCHAR(10) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Entidad JPA: Estudiante.java

- `id` (Long) - PK auto-generada
- `run` (String) - RUN chileno, identificador único
- `nombre` (String) - Nombre del estudiante
- `curso` (String) - Curso asignado (ej: "1-A", "3-B")

asistencia-service - Tabla asistencias

```
CREATE TABLE asistencias (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  estudiante_id BIGINT NOT NULL,
  fecha DATE NOT NULL,
  presente BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE asistencias ADD INDEX idx_estudiante (estudiante_id);
```

Entidad JPA: Asistencia.java

- `id` (Long) - PK auto-generada
- `estudianteId` (Long) - FK referencia a estudiante (sin constraint de BD)
- `fecha` (String/LocalDate) - Fecha de asistencia
- `presente` (boolean) - Estado de asistencia

evaluacion-service - Tabla evaluaciones

```
CREATE TABLE evaluaciones (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  estudiante_id BIGINT NOT NULL,
  materia VARCHAR(100) NOT NULL,
  nota DECIMAL(5, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

ALTER TABLE evaluaciones ADD INDEX idx_estudiante (estudiante_id);
```

Entidad JPA: Evaluacion.java

- `id` (Long) - PK auto-generada
- `estudianteId` (Long) - FK referencia a estudiante (sin constraint de BD)
- `materia` (String) - Nombre de la materia
- `nota` (Double) - Nota numérica (0.0 - 10.0)

Configuración de base de datos

Variables de entorno requeridas

Todos los microservicios usan estas variables para configurar la conexión:

```
SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/nombre_db
SPRING_DATASOURCE_USERNAME=usuario_db
SPRING_DATASOURCE_PASSWORD=contraseña_db
SPRING_JPA_DATABASE_PLATFORM=org.hibernate.dialect.MySQL8Dialect
SPRING_JPA_HIBERNATE_DDL_AUTO=update
```

application.properties de cada servicio

La configuración se centraliza con ENV vars:

```
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.database-platform=${SPRING_JPA_DATABASE_PLATFORM}
spring.jpa.hibernate.ddl-auto=${SPRING_JPA_HIBERNATE_DDL_AUTO}
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
```

Estrategia de migraciones

Herramienta: Hibernate DDL-Auto (configurado en `spring.jpa.hibernate.ddl-auto`)

- `update`: Crea tablas si no existen y actualiza esquema (usado en desarrollo/test)
- `create`: Crea nuevo esquema cada vez (solo para testing)
- `validate`: Valida esquema sin cambios (producción recomendado)

Alternativa: Se recomienda usar **Flyway** o **Liquibase** para producciones con versionado de migrations.

Consideraciones de rendimiento y escalado

Índices actuales

```
-- Tabla asistencias
CREATE INDEX idx_asistencia_estudiante ON asistencias(estudiante_id);

-- Tabla evaluaciones
CREATE INDEX idx_evaluacion_estudiante ON evaluaciones(estudiante_id);
```

Pool de conexiones (HikariCP)

Configuración recomendada en `application.properties`:

```
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=20000
```

Escalabilidad futura

- **Sharding horizontal:** Por CURSO (asistencias y evaluaciones)
- **Replicación:** Slaves de lectura para estudiantes (tabla más consultada)

- **Caché distribuido:** Redis para datos de estudiantes frecuentes

Integridad referencial

Nota: Las relaciones entre tablas (estudiantes ↔ asistencias/evaluaciones) se validan en la **capa de aplicación** en lugar de usar Foreign Keys, permitiendo:

- Mayor flexibilidad en escalado de microservicios
- Evitar dependencias de BD entre servicios
- Fallos aislados sin cascada

Backup y recuperación

Política recomendada

- **Frecuencia:** Diaria (automática en producción)
- **Destino:** Cloud storage (ej: AWS S3, Google Cloud Storage)
- **Retención:** 30 días mínimo

Procedimiento de restauración

```
# Respaldar BD actual
mysqldump -u usuario -p nombre_db > backup_$(date +%Y%m%d_%H%M%S).sql

# Restaurar desde backup
mysql -u usuario -p nombre_db < backup_YYYYMMDD_HHMMSS.sql
```

Anexos

Ejemplo de Entity JPA

```
@Entity
@Table(name = "estudiantes")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Estudiante {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String run;

    @Column(nullable = false)
    private String nombre;

    @Column(nullable = false)
    private String curso;
}
```